

Universidad Sergio Arboleda

Ciencia De La Computacion e Inteligencia Artificial — Metricas De Rendimiento

Taller Practico 3

Procesamiento de Video con CUDA:
Deteccion de Trayectoria de Asteroides
CPU vs GPU

Estudiante: Antony Camilo Barahona Pena

Curso: Metricas De Rendimiento

Universidad: Universidad Sergio Arboleda

Fecha: Abril 2026

Implementado en Python 3.12 — CUDA 12.8 — Numba 0.60.0
Ejecutado en Google Colaboratory con GPU NVIDIA Tesla T4

Contents

1	Titulo y Objetivos	2
1.1	Titulo	2
1.2	Objetivo General	2
1.3	Objetivos Especificos	2
2	Marco Teorico	2
2.1	Procesamiento de Video como Secuencia de Imagenes	2
2.2	Conversion a Escala de Grises	2
2.3	Filtro de Suavizado Espacial (Blur 3x3)	3
2.4	Diferenciacion Temporal (Sustraccion de Fondo)	3
2.5	Umbralización Binaria	3
2.6	Morfologia Matematica: Erosion y Dilatacion	3
2.7	Computacion Paralela con CUDA	4
2.8	Numba CUDA para Python	4
3	Metodologia	4
3.1	Configuracion del Hardware	4
3.2	Configuracion del Software	4
3.3	Generacion del Video Sintetico	5
3.4	Descripcion del Pipeline de Deteccion	6
3.4.1	Etapas 1: Conversion a Escala de Grises	6
3.4.2	Etapas 2: Filtro de Suavizado 3x3	6
3.4.3	Etapas 3: Diferenciacion Temporal	6
3.4.4	Etapas 4: Umbralización Binaria	7
3.4.5	Etapas 5: Erosion y Dilatacion (Apertura Morfologica)	7
3.4.6	Etapas 6: Extraccion de Centroides y Filtro de Consistencia	8
4	Resultados	8
4.1	Veredicto de Deteccion	8
4.2	Comparativa de Rendimiento	8
5	Analisis de Rendimiento	8
5.1	Por que la CPU tarda tanto: analisis tecnico	9
5.2	Por que la GPU es tan rapida	9
5.3	Robustez del Algoritmo de Deteccion	10
6	Conclusiones	10

1 Titulo y Objetivos

1.1 Titulo

Procesamiento de video en algoritmos CUDA con GPU vs CPU, detectando trayectoria de asteroides.

1.2 Objetivo General

Desarrollar e implementar un algoritmo de vision artificial capaz de detectar automaticamente y rastrear la trayectoria de un asteroide sobre un fondo estelar ruidoso, comparando cuantitativamente el rendimiento de dos implementaciones: una secuencial en CPU y una paralela en GPU mediante CUDA.

1.3 Objetivos Especificos

- Implementar seis etapas de procesamiento de vision artificial como kernels CUDA escritos manualmente: conversion a escala de grises, suavizado espacial, diferenciacion temporal, umbralización, erosion y dilatacion morfologica.
- Replicar exactamente el mismo pipeline de forma secuencial en CPU usando Python puro (sin vectorizacion con NumPy) para garantizar una comparativa justa del esfuerzo computacional.
- Medir y analizar el speedup obtenido al trasladar la carga de trabajo a la GPU, justificando tecnica y matematicamente la diferencia de rendimiento.
- Generar un video sintetico aleatorio y realista que simule condiciones variables de observacion espacial, con y sin presencia de asteroide.

2 Marco Teorico

2.1 Procesamiento de Video como Secuencia de Imagenes

Un video digital es una secuencia de imagenes estaticas (frames) capturadas a una tasa fija expresada en fotogramas por segundo (fps). Cada frame es una matriz tridimensional de dimensiones $H \times W \times C$, donde H es la altura en pixeles, W el ancho y C el numero de canales de color (tipicamente 3 para RGB). Procesar un video implica aplicar transformaciones a cada uno de estos frames de forma ordenada en el tiempo.

2.2 Conversion a Escala de Grises

La conversion de una imagen a color RGB a escala de grises reduce la dimensionalidad de $H \times W \times 3$ a $H \times W$, eliminando la informacion de crominancia y conservando unicamente la luminancia. La formula estandar ponderada ITU-R BT.601 es:

$$I_{gray}(i, j) = 0.2989 \cdot R(i, j) + 0.5870 \cdot G(i, j) + 0.1140 \cdot B(i, j) \quad (1)$$

Los coeficientes reflejan la sensibilidad perceptual del ojo humano: mayor sensibilidad al verde, moderada al rojo y menor al azul.

2.3 Filtro de Suavizado Espacial (Blur 3x3)

Un filtro de suavizado promedio recorre la imagen pixel por pixel y reemplaza cada valor por el promedio de su vecindad local. Para un kernel de 3×3 :

$$I_{out}(i, j) = \frac{1}{|N|} \sum_{(ni, nj) \in N(i, j)} I_{in}(ni, nj) \quad (2)$$

donde $N(i, j)$ es el conjunto de pixeles dentro del bloque 3×3 centrado en (i, j) . Este filtro reduce el ruido de sal y pimienta (pixeles aleatorios brillantes) sin eliminar estructuras de mayor tamaño como el asteroide.

2.4 Diferenciación Temporal (Sustracción de Fondo)

La sustracción de fondo es una técnica clásica de detección de movimiento. Se calcula como la diferencia en valor absoluto entre el frame actual y un frame anterior separado por un intervalo de tiempo Δt :

$$D(i, j) = |I_t(i, j) - I_{t-\Delta t}(i, j)| \quad (3)$$

El fondo estelar fijo produce $D \approx 0$. Un objeto en movimiento produce $D > 0$ proporcional a su velocidad aparente. En este taller se utilizó $\Delta t = 4$ frames para detectar asteroides de velocidad lenta, ya que con $\Delta t = 1$ el desplazamiento entre frames consecutivos es insuficiente para generar una diferencia detectable por encima del umbral de ruido.

2.5 Umbralización Binaria

Tras la diferenciación temporal, la imagen resultante contiene valores continuos. La umbralización convierte estos valores a una imagen binaria:

$$B(i, j) = \begin{cases} 255 & \text{si } D(i, j) \geq \tau \\ 0 & \text{en otro caso} \end{cases} \quad (4)$$

El parámetro τ (umbral) se calibró experimentalmente en $\tau = 30.0$ para el video sintético generado, equilibrando la sensibilidad ante el asteroide y la supresión de falsas detecciones por ruido.

2.6 Morfología Matemática: Erosion y Dilatación

La morfología matemática opera sobre imágenes binarias usando un elemento estructural (kernel). Las dos operaciones fundamentales son:

Erosion: Un pixel permanece blanco (255) solo si *todos* los pixeles de su vecindad 3×3 son blancos. Elimina objetos pequeños aislados (ruido).

$$E(i, j) = \min_{(di, dj) \in [-1, 1]^2} B(i + di, j + dj) \quad (5)$$

Dilatacion: Un pixel se convierte en blanco si *al menos uno* de los pixeles de su vecindad es blanco. Expande las regiones blancas.

$$D_{mor}(i, j) = \max_{(di, dj) \in [-1, 1]^2} E(i + di, j + dj) \quad (6)$$

La combinacion erosion seguida de dilatacion se denomina *apertura morfologica* (Opening) y logra eliminar el ruido residual preservando objetos de tamano suficiente como el asteroide.

2.7 Computacion Paralela con CUDA

CUDA (Compute Unified Device Architecture) es el modelo de programacion paralela de NVIDIA para GPUs. Su arquitectura organiza la ejecucion en una jerarquia de tres niveles:

- **Grid:** Conjunto de todos los bloques de hilos que ejecutan un kernel.
- **Bloque (Block):** Grupo de hilos que comparten memoria compartida y pueden sincronizarse.
- **Hilo (Thread):** Unidad minima de ejecucion, procesa un elemento de datos.

En el procesamiento de imagenes, la estrategia natural es asignar un hilo CUDA a cada pixel. Para una imagen de 480×640 pixeles, se lanzan $480 \times 640 = 307.200$ hilos simultaneos, cada uno calculando independientemente su valor de salida.

La configuracion de bloques e hilos utilizada en este taller fue:

$$\text{threadsperblock} = (16, 16) \Rightarrow 256 \text{ hilos por bloque} \quad (7)$$

$$\text{blockspergrid} = \left(\left\lceil \frac{W}{16} \right\rceil, \left\lceil \frac{H}{16} \right\rceil \right) = (40, 30) \Rightarrow 1.200 \text{ bloques} \quad (8)$$

2.8 Numba CUDA para Python

Numba es un compilador JIT (Just-In-Time) para Python que permite escribir kernels CUDA directamente en Python usando el decorador `@cuda.jit`. El codigo Python es compilado en tiempo de ejecucion a PTX (codigo intermedio de NVIDIA), eliminando la necesidad de escribir C++ CUDA. La libreria `pynvjitlink-cu12` resuelve la integracion entre Numba y el linker de CUDA 12.x.

3 Metodologia

3.1 Configuracion del Hardware

3.2 Configuracion del Software

La libreria `pynvjitlink-cu12` es requerida por Numba 0.60.0 para enlazar correctamente los kernels compilados con CUDA 12.x, resolviendo una incompatibilidad de versiones entre el linker `nvjitlink` y versiones anteriores del compilador PTX de Numba.

Componente	Especificacion
Plataforma	Google Colaboratory
GPU	NVIDIA Tesla T4
Memoria GPU	15.360 MiB (GDDR6)
CUDA Cores	2.560
Arquitectura GPU	Turing (sm_75)
CPU	Intel Xeon (asignacion Colab)
RAM Sistema	12 GB aprox.

Table 1: Especificaciones del hardware utilizado.

Componente	Version
Sistema Operativo	Ubuntu 22.04 LTS
Python	3.12.13
CUDA Toolkit	12.8 (V12.8.93)
Numba	0.60.0
pynvjitlink	cu12
NumPy	2.0.2
OpenCV	4.13.0
Matplotlib	(version Colab)

Table 2: Especificaciones del software utilizado.

3.3 Generacion del Video Sintetico

Dado que el taller no proporciono acceso publico a los videos de prueba originales, se implemento un generador de video sintetico que simula condiciones realistas de observacion espacial. Las características del video generado son:

- **Resolucion:** 640×480 pixeles, 90 frames a 30 fps.
- **Fondo:** Gradiente espacial oscuro, nebulosas difusas de posicion y color aleatorios, y entre 180 y 280 estrellas por frame de posicion y brillo completamente aleatorios (parpadeantes) para simular ruido atmosferico y movimiento de camara.
- **Ruido uniforme:** Capa de ruido aditivo aleatorio de hasta 20 unidades de intensidad por canal.
- **Asteroide (opcional):** Objeto circular multicapa (halo + cuerpo + nucleo + reflejo de luz) de radio 18 pixeles, presente o ausente de forma aleatoria con probabilidad 50%.
- **Trayectorias posibles:** Lineal horizontal, diagonal inversa, curva sinusoidal, cambio de rumbo (quiebre a mitad del recorrido) y zigzag.
- **Velocidades posibles:** Lenta ($\times 0.4$), normal ($\times 1.0$) y rapida ($\times 2.2$).

El video se recodifica con `ffmpeg` usando el codec H.264 para compatibilidad de reproduccion en el entorno Colab.

3.4 Descripción del Pipeline de Detección

El pipeline implementa seis etapas secuenciales de procesamiento. Cada etapa fue implementada dos veces: una como kernel CUDA (GPU) y otra como función con loops puros en Python (CPU).

3.4.1 Etapa 1: Conversión a Escala de Grises

Cada frame BGR de OpenCV se convierte a RGB y luego a float32 en rango $[0, 255]$. El kernel CUDA asigna un hilo por pixel para calcular la luminancia según la fórmula ITU-R BT.601.

Listing 1: Kernel CUDA de conversión a escala de grises.

```

1 @cuda.jit
2 def grayscale_kernel(frame_color, frame_gray):
3     j, i = cuda.grid(2)
4     rows, cols = frame_color.shape
5     if i < rows and j < cols:
6         R = frame_color[i, j, 0]
7         G = frame_color[i, j, 1]
8         B = frame_color[i, j, 2]
9         frame_gray[i, j] = 0.2989 * R + 0.5870 * G + 0.1140 * B

```

3.4.2 Etapa 2: Filtro de Suavizado 3x3

El kernel recorre cada pixel y calcula el promedio de su vecindad 3×3 , manejando los casos de borde verificando los límites de la imagen dentro del propio hilo.

Listing 2: Kernel CUDA de suavizado espacial.

```

1 @cuda.jit
2 def filtro_suavizado_kernel(frame_in, frame_out):
3     j, i = cuda.grid(2)
4     rows, cols = frame_in.shape
5     if i < rows and j < cols:
6         suma = 0.0
7         count = 0
8         for di in range(-1, 2):
9             for dj in range(-1, 2):
10                 ni, nj = i+di, j+dj
11                 if 0 <= ni < rows and 0 <= nj < cols:
12                     suma += frame_in[ni, nj]
13                     count += 1
14                 frame_out[i, j] = suma / count

```

3.4.3 Etapa 3: Diferenciación Temporal

Se compara cada frame con el frame ubicado 4 posiciones atrás en el tiempo ($\Delta t = 4$). Esta decisión se tomó porque con $\Delta t = 1$ los asteroides de velocidad lenta generan diferencias imperceptibles por encima del umbral de ruido. Al ampliar el intervalo temporal se acumula suficiente desplazamiento para que incluso objetos lentos sean detectables.

Listing 3: Kernel CUDA de diferenciación temporal.

```

1 @cuda.jit
2 def diferencia_temporal_kernel(frame_actual, frame_anterior,
3     frame_diff):
4     j, i = cuda.grid(2)
5     rows, cols = frame_actual.shape
6     if i < rows and j < cols:
7         frame_diff[i, j] = abs(frame_actual[i, j] - frame_anterior[i,
8             j])

```

3.4.4 Etapa 4: Umbralización Binaria

Se aplica un umbral $\tau = 30.0$ calibrado experimentalmente. Este valor fue determinado mediante pruebas iterativas: valores menores generaban miles de falsos positivos por el ruido de estrellas parpadeantes; valores mayores eliminaban la señal del asteroide en trayectorias lentas.

3.4.5 Etapa 5: Erosion y Dilatacion (Apertura Morfologica)

Se aplican erosion seguida de dilatacion, ambas con kernel 3×3 . La erosion elimina pixeles blancos aislados (ruido residual de 1-2 pixeles). La dilatacion posterior restaura el tamaño original del asteroide que fue reducido por la erosion.

Listing 4: Kernels CUDA de erosion y dilatacion.

```

1 @cuda.jit
2 def erosion_kernel(frame_in, frame_out):
3     j, i = cuda.grid(2)
4     rows, cols = frame_in.shape
5     if i < rows and j < cols:
6         minimo = 255.0
7         for di in range(-1, 2):
8             for dj in range(-1, 2):
9                 ni, nj = i+di, j+dj
10                if 0 <= ni < rows and 0 <= nj < cols:
11                    if frame_in[ni, nj] < minimo:
12                        minimo = frame_in[ni, nj]
13            frame_out[i, j] = minimo
14
15 @cuda.jit
16 def dilatacion_kernel(frame_in, frame_out):
17     j, i = cuda.grid(2)
18     rows, cols = frame_in.shape
19     if i < rows and j < cols:
20         maximo = 0.0
21         for di in range(-1, 2):
22             for dj in range(-1, 2):
23                 ni, nj = i+di, j+dj
24                 if 0 <= ni < rows and 0 <= nj < cols:
25                     if frame_in[ni, nj] > maximo:
26                         maximo = frame_in[ni, nj]
27            frame_out[i, j] = maximo

```


3.4.6 Etapa 6: Extraccion de Centroide y Filtro de Consistencia

Se implemento un algoritmo de deteccion de centroide basado en densidad local en bloques de 32×32 pixeles. La logica es la siguiente:

1. Dividir el frame en bloques de 32×32 .
2. Contar pixeles blancos en cada bloque.
3. Si el bloque de mayor densidad supera el 15% de su area (≥ 154 pixeles), se considera deteccion valida.
4. Calcular el centroide (x, y) como la media de los pixeles blancos dentro del bloque mas denso.

Esta estrategia es superior al simple conteo de pixeles blancos porque el ruido genera pixeles *dispersos* en toda la imagen (densidad maxima por bloque: ≈ 90 – 120 pixeles), mientras que el asteroide genera una *concentracion* de pixeles en una zona reducida (densidad: ≈ 280 – 800 pixeles).

Adicionalmente, se aplica un filtro de consistencia temporal: solo se registra una deteccion como valida si aparece en al menos 5 frames consecutivos, eliminando los falsos positivos espurios que el ruido pudo haber generado de forma aislada.

4 Resultados

4.1 Veredicto de Deteccion

El algoritmo produce dos salidas booleanas: una por la version GPU y otra por la version CPU. En todas las pruebas realizadas, ambas versiones produjeron el mismo veredicto, confirmando la equivalencia funcional de las implementaciones.

Escenario	Trayectoria	Velocidad	GPU	CPU
Con asteroide	curva	rapida	EXISTE	EXISTE
Con asteroide	zigzag	lenta	EXISTE	EXISTE
Con asteroide	cambio_rumbo	normal	EXISTE	EXISTE
Sin asteroide	—	—	NO EXISTE	NO EXISTE
Sin asteroide	—	—	NO EXISTE	NO EXISTE

Table 3: Resultados de deteccion en distintos escenarios de prueba.

4.2 Comparativa de Rendimiento

Las mediciones de tiempo se realizaron sobre el mismo video de 90 frames a 640×480 pixeles, incluyendo todas las etapas del pipeline: lectura de frames, conversion a gris, suavizado, diferenciacion, umbralización, morfologia y extraccion de centroide.

5 Analisis de Rendimiento

Metrica	CPU Secuencial	GPU CUDA
Tiempo total	$\approx 928\text{--}942\text{ s}$	$\approx 1.94\text{--}2.04\text{ s}$
Frames procesados	90	90
Speedup	$1\times$ (referencia)	$\approx 461\text{--}478\times$
Reduccion de tiempo	0%	99.8%

Table 4: Comparativa de rendimiento CPU vs GPU.

5.1 Por que la CPU tarda tanto: analisis tecnico

La implementacion CPU secuencial replica exactamente la misma logica matematica que los kernels CUDA, pero ejecutada con loops anidados en Python puro. Esta decision fue *intencional y justificada* por los siguientes motivos:

- 1. Equivalencia funcional:** Para que la comparativa sea valida, ambas implementaciones deben realizar exactamente el mismo computo. Usar funciones vectorizadas de NumPy en la CPU habria sido una comparativa injusta, ya que NumPy en si mismo utiliza instrucciones SIMD (AVX/SSE) y multiples nucleos internamente.
- 2. Reflejo del modelo de programacion:** Los kernels CUDA operan pixel por pixel (un hilo = un pixel). El equivalente directo en CPU secuencial es un loop que recorre pixel por pixel. Esta correspondencia uno a uno hace la comparativa tecnica y pedagogicamente honesta.
- 3. Costo computacional real:** Para cada frame de $480 \times 640 = 307.200$ pixeles, el suavizado 3×3 requiere hasta 9 accesos de memoria y operaciones aritmeticas por pixel. La erosion y dilatacion idem. En Python puro, cada iteracion del loop tiene un overhead de interpretacion del orden de microsegundos. Para 90 frames y 3 operaciones con loops (suavizado, erosion, dilatacion):

$$\text{Operaciones} \approx 3 \times 90 \times 307.200 \times 9 \approx 745 \text{ millones de operaciones} \quad (9)$$

Python interpreta aproximadamente 10–50 millones de operaciones simples por segundo, lo que justifica los ≈ 930 segundos observados.

5.2 Por que la GPU es tan rapida

La GPU NVIDIA Tesla T4 cuenta con 2.560 nucleos CUDA que operan en paralelo. En lugar de procesar los 307.200 pixeles de forma secuencial, los procesa todos *al mismo tiempo* en grupos de 256 hilos por bloque.

El factor de aceleracion teorico maximo esta acotado por la Ley de Amdahl. En este pipeline, practicamente el 100% del computo es paralelizable (cada pixel es independiente de los demas dentro de cada kernel), por lo que el speedup observado de $\approx 461\times$ refleja la diferencia real en capacidad de computo entre la CPU de proposito general y la GPU especializada en operaciones masivamente paralelas.

Adicionalmente, el modelo de memoria CUDA optimiza la transferencia entre la memoria del host (RAM) y el dispositivo (VRAM): `cuda.to_device()` copia el frame a la VRAM

antes del computo, y `copy_to_host()` lo recupera solo cuando es necesario, minimizando la latencia de comunicacion PCIe.

5.3 Robustez del Algoritmo de Deteccion

El principal desafio tecnico del taller fue diseniar un algoritmo capaz de distinguir el asteroide del ruido de fondo, siendo que el fondo tambien cambia entre frames (estrellas parpadeantes). Se evaluaron y descartaron las siguientes aproximaciones:

1. **Conteo simple de pixeles blancos:** Fallo porque el ruido genera miles de pixeles blancos dispersos en toda la imagen, indistinguibles en cantidad del asteroide.
2. **Umbral bajo ($\tau < 12$):** Genero miles de pixeles blancos por frame incluso sin asteroide.
3. **Umbral alto ($\tau > 50$):** Elimino la senal del asteroide en trayectorias lentas.
4. **Diferencia con frame anterior ($\Delta t = 1$):** Insuficiente para asteroides lentos cuyo desplazamiento entre frames consecutivos es menor al ruido.

La solucion final combina tres mecanismos complementarios:

- Umbral $\tau = 30$ para separar senal de ruido en la imagen diferencia.
- Analisis de densidad local en bloques 32×32 (umbral 15% de ocupacion) para distinguir concentracion (asteroide) de dispersion (ruido).
- Filtro de consistencia temporal de 5 frames consecutivos para eliminar detecciones espurias aisladas.

6 Conclusiones

Sobre la computacion paralela: La GPU NVIDIA Tesla T4 proceso el pipeline completo de 90 frames en aproximadamente 2 segundos, frente a los 930 segundos de la CPU secuencial, logrando un speedup de $\approx 461\times$. Esta diferencia no es un artefacto del lenguaje ni del entorno: es la consecuencia directa de ejecutar 2.560 nucleos en paralelo frente a un solo nucleo secuencial en Python.

Sobre los kernels CUDA: Escribir los kernels manualmente (en lugar de usar librerias de alto nivel como CuPy) obliga a comprender y gestionar explicitamente la jerarquia de hilos, bloques y grids, la configuracion del lanzamiento y el manejo de los limites de la imagen. Este nivel de control es fundamental en aplicaciones de alto rendimiento donde el acceso a memoria y la ocupacion de la GPU deben optimizarse manualmente.

Sobre el algoritmo de deteccion: El analisis de densidad local en bloques resulto ser la clave para filtrar el ruido de fondo, superando las limitaciones del umbral simple. La combinacion de diferenciacion temporal con salto de 4 frames, umbralización, apertura morfologica y consistencia temporal produjo un detector robusto ante distintas trayectorias y velocidades.

Sobre el diseno experimental: La decision de implementar la CPU con loops puros en Python, replicando exactamente la logica del kernel CUDA, garantiza que la comparativa refleje fielmente la diferencia entre el modelo secuencial y el paralelo,

sin intervenciones de optimizaciones ocultas como SIMD o multithreading implicito de NumPy.

Aprendizaje principal: El procesamiento de video es un dominio naturalmente paralelo: cada pixel de cada frame puede procesarse de forma independiente. Esta independencia de datos es exactamente la condicion que la arquitectura CUDA explota al maxima, lo que convierte a las GPUs en la plataforma ideal para vision artificial en tiempo real.